

Diretrizes Gerais da AEP do 4º Semestre (2026.2)

- A Atividade de Estudo Programada (AEP) é um trabalho interdisciplinar obrigatório do curso de Engenharia de Software, desenvolvido em equipes de até três integrantes, com o objetivo de integrar os conhecimentos adquiridos nas disciplinas do semestre por meio do desenvolvimento incremental de uma solução computacional.
- **Cursos Integrados:** Engenharia de Software (Esoft) e Análise e Desenvolvimento de Sistemas (ADS).
- **Estratégia:** Uma única AEP será desenvolvida para ambos os cursos, visando a otimização de tempo e esforço.
- **Regra de Escopo:** Para garantir a equidade, os requisitos não podem cobrar (ao menos explicitamente) entregas baseadas em disciplinas exclusivas de apenas um dos cursos.
- **Temática Obrigatória:** O projeto desenvolvido pelos alunos deve estar obrigatoriamente alinhado a um ou mais **Objetivos de Desenvolvimento Sustentável (ODS) da ONU**.
- **Pontuação:** A atividade valerá **1,0 ponto na média final de cada bimestre**, sendo essa nota aplicada a todas as disciplinas envolvidas.
- **CrITÉrios de Avaliação:** Os critérios de correção e as métricas de pontuação serão elaborados utilizando um sistema de múltiplos agentes.

I. Disciplinas Envolvidas

Disciplina	Cursos Atendidos
Banco de Dados	Esoft e ADS
Programação Orientada a Objetos	Esoft e ADS
Sistemas Operacionais	Esoft e ADS
Engenharia de Requisitos	Apenas Esoft
Gestão de Projetos	Apenas ADS
ETI	Apenas ADS

II. Cronograma e Requisitos de Entrega

O projeto contará com duas grandes entregas, uma ao final de cada bimestre.

1ª Entrega (Primeiro Bimestre)

O foco desta etapa é a concepção do projeto, o planejamento da arquitetura e a estruturação do ambiente.

- **Formato:** Um documento em PDF formal e bem formatado.
- **Conteúdo do Documento:**
 - **Descoberta:** descrição dos principais requisitos identificados na pesquisa e levantamento, e que serviram de inspiração ou origem para o projeto. **ATENÇÃO** na entrega de valor para as partes interessadas e regras de negócios!! Identificar e focar nos principais problemas (dores) e suas origens.
 - **Concepção e Alinhamento:** descrição do projeto e explicação clara de como ele soluciona problemas relacionados aos ODS da ONU.
 - **Justificativa Técnica e Arquitetural:** defesa das escolhas do projeto. Os alunos devem explicar o porquê de utilizarem determinada arquitetura, linguagem, banco de dados e demais ferramentas tecnológicas.
 - **Ambiente de Controle de Versão:** o PDF deve conter o link de acesso ao repositório do projeto no GitHub.
 - **Estrutura do Repositório:** o GitHub deve estar com toda a estrutura de diretórios do projeto previamente criada e organizada de forma lógica.
 - **Diagrama de Classe:** Deve conter todos os relacionamentos com ao menos uma composição de multiplicidade 1:N, Herança e Polimorfismo.
 - **Diagrama do Banco:** Diagrama do banco o DER.
- **Atenção:** não é necessária a entrega de código funcional nesta fase, apenas a estruturação e o projeto arquitetural.

2ª Entrega (Segundo Bimestre)

O foco desta etapa é a entrega final do software funcional e integrado.

- **Conteúdo:** código-fonte completo da aplicação.
- **Requisitos Técnicos:**
 - Aplicação prática dos pilares da Programação Orientada a Objetos (herança, composição, polimorfismo, etc.).
 - Desenvolvimento de operações de CRUD completas.
 - Conexão ativa e integração com um banco de dados.
 - Contribuição de TODOS no repositório.

III. Fundamentação para Atribuição de Nota Baixa

Este documento foi estruturado para que o professor tenha **evidência documental** da falha do aluno:

1. **Falta de Requisitos:** se o aluno entregar um PDF "bonitinho" mas sem a lista dos requisitos (escopo), ele perde 0,2 de imediato.
2. **Falta de Cronograma:** Se não houver a tabela de datas e responsáveis, o aluno demonstra incapacidade de autogestão, perdendo 0,2.
3. **Inconsistência:** se na 2ª entrega o aluno apresentar algo diferente do que planejou na 1ª, o professor tem base para dizer: *"Você não cumpriu o contrato de desenvolvimento que você mesmo escreveu no cronograma e requisitos"*, descontando o peso de "Aderência ao Planejamento".
4. **Simulação de CRUD:** se o banco de dados não estiver conectado (usar listas em memória), a nota de persistência é zero (0,2), sem meio-certo.

IV. Sistema de Pontuação da AEP

1ª Entrega: Planejamento, Escopo e Estruturação (Bimestre 1)

Critério	Métrica de Aceitação (O que deve conter)	Peso
Escopo Funcional e ODS	Descrição do software alinhada à ODS. Deve conter uma Lista de Requisitos Funcionais (excluir “funcionais”) (mínimo 5 itens) descrevendo exatamente o que o sistema (a parte interessada ou a regra de negócio identificada trouxe como necessidade ou dor, e escrita como o exemplo abaixo: Ex: O sistema deve permitir o cadastro de doadores, com nome completo, idade, endereço, celular, email, endereço e tipo sanguíneo.	0,2
Cronograma de Execução	Uma tabela no PDF e no README do GitHub contendo as etapas de desenvolvimento (Datas, Atividade e Responsável). O não cumprimento de prazos simulados no cronograma não tira nota agora, mas a ausência da tabela zera o critério.	0,2
Justificativa Técnica	Defesa da Linguagem, Banco de Dados e Padrão Arquitetural. Deve explicar por que essas escolhas suportam o Escopo Funcional (excluir) definido acima.	0,3
Estrutura de GitHub	Link público. Pastas: /src, /docs, /database. O arquivo README.md deve obrigatoriamente espelhar o Cronograma e a Lista de Requisitos do PDF.	0,3

- **Nota Zero Automática na 1ª Entrega se:** Ausência da lista de requisitos, ausência do cronograma ou link do GitHub inválido/privado.

2ª Entrega: Implementação, Rigor e Integração (Bimestre 2)

Critério	Métrica de Aceitação Algorítmica/Visual	Peso
Aderência ao Planejamento do Projeto	O código entregue deve refletir os Requisitos Funcionais (excluir) e funcionalidades listados na 1ª entrega. Se o aluno planejou um "CRUD de usuários" e entregou um "Cadastro de livros", perde-se este ponto por inconsistência de projeto.	0,2
Pilares de POO	Uso obrigatório de: 1) Classe Abstrata ou Interface; 2) Herança; 3) Polimorfismo via @Override. O código deve ser tipado e seguir convenções (CamelCase).	0,3
CRUD Funcional	Operações de Create, Read, Update e Delete operando sobre a entidade principal definida no escopo da 1ª entrega.	0,3
Persistência em BD	Conexão ativa com Banco de Dados. O script SQL de criação das tabelas deve estar na pasta /database e deve ser compatível com as classes Java/C#.	0,2

- **Nota Zero Automática na 2ª Entrega se:** O software não executar, se os pilares de POO forem ignorados (código procedural) ou se a entrega divergir 100% do planejado no 1º Bimestre.

Orientações Finais aos Alunos

1. **Independência de Disciplinas:** Não será cobrada interface gráfica avançada (ETI). A nota foca exclusivamente em: **O projeto está fundamentado tecnicamente? O código funciona? Ele é orientado a objetos? Ele salva no banco?**
2. **Objetividade na Correção:** A ausência de um dos pilares de POO (ex: não usou Herança) acarretará na perda total dos 0,4 pontos do critério, independente de o software "estar bonitinho".
3. **Ambiente de Sistemas Operacionais:** O projeto deve conter instruções claras de como executar (instalação de dependências), respeitando a compatibilidade de ambientes discutida na disciplina de SO.
4. **Plágio:** Projetos com estrutura de classes, nomes de variáveis e contexto idênticos entre grupos serão zerados sem possibilidade de recurso, conforme regimento institucional.

Este documento serve como contrato de entrega. A não observância de qualquer métrica acima implica em redução imediata da nota conforme os pesos estabelecidos.